

Analyzer Coder.

让项目知识， 回到代码现场。

整个项目结项报告。将源码与业务知识汇入同一检索入口，让每次查找都有来源，每条证据都有版本。

● 阶段性研发结项

2026年9月15日

业务与生产验收待确认



DISCOVERY / 查找

实现与约束，一起找到

从业务术语出发，同时定位源码片段与相关知识。

PROVENANCE / 核对

答案有依据，证据有版本

保留原文、行号、修订和快照，支持人工回查。

EVOLUTION / 演进

主流程收敛，质量继续提高

完成核心研发交付，下一步补齐量化评测与运行验收。

01 项目概述

Analyzer Coder 是面向开发者与项目维护人员的代码与知识检索平台。项目将源码、项目说明和人工维护的知识组织为有来源、可定位、可区分版本的检索资产，让用户从业务问题出发找到相关实现与约束，并通过原始材料核对答

案。

当前主流程为：接入项目 → 选择并准备分支快照 → 维护关联知识 → 联合检索 → 查看源码和知识来源。项目问答、代码图谱及 MCP 在这条主流程上提供辅助能力。

项目定位已经从较宽的工程分析平台收敛为代码与知识联合检索工具。跨仓工程项目、变更审查、PR/MR Webhook、审查 CI、任务结果回报等能力已退出当前产品范围，不作为本次结项的现行交付成果。

02 建设背景与解决的问题

2.1 建设背景

在项目研发和维护过程中，代码实现、架构说明、业务约束与开发经验通常分散在仓库文件、Markdown 和个人经验中。传统文件搜索偏向路径和关键词，开发者即使找到代码，也可能缺少理解该实现所需的业务背景。引入 AI 后，如果答案没有可靠来源或混用了不同版本的材料，又会增加核对成本。

随着项目采用多分支开发，同一文件或知识在不同分支上的含义与有效性可能不同。因此，系统需要同时解决信息查找、来源核对、版本隔离和知识维护的问题。

2.2 主要问题与项目应对

背景问题	项目提供的应对能力	实际价值与边界
只知道业务术语，不知道对应代码位置	联合检索源码片段与项目知识，提供关键词及可选向量通道	提供从业务描述到实现的查找入口；复杂问题的召回效果仍需持续评测
代码和文档分散，实现与约束难以同时找到	知识卡绑定路径、符号等范围，检索结果同时展示代码和知识	减少在不同材料之间反复查找；依赖知识录入质量与关联准确性
AI 回答缺少可核对的依据	问答保留引用，结果可回到源码位置、知识卡及其来源	支持人工核对；有引用不等于事实已经被证明
多分支代码与知识容易混用	阅读上下文绑定项目、分支、快照与提交，查询限定相应范围	避免用其他分支或旧索引冒充当前结果；未准备时明确阻断或降级
共享知识和分支专属知识容易相互覆盖	共享与专属知识作为独立卡片维护，分别保留身份、修订与发布记录	保留共性经验和分支差异；冲突内容仍需人工判断
代码变化后，知识是否仍有效不明确	来源漂移提示与分支验证，验证绑定知识修订、分支及快照	让有效性核验有记录；系统不自动证明业务规则正确
功能入口过多，主流程不突出	清理遗留功能、简化分支面板、统一菜单命名并集中任务历史	降低理解界面的负担；真实用户体验仍需使用反馈验证

上述收益是当前设计与能力所支持的预期价值。本项目尚无足够的用户统计数据证明具体节省了多少时间或提升了多少效率。

03 主要建设成果

3.1 项目接入与多分支管理

支持本地 Git、远程 Git 及 ZIP 接入。一个 Git 项目对应一个仓库逻辑身份，可管理多个分支；分支同步和索引产物以受管快照组织，不依靠 checkout 用户源工作区来切换阅读版本。ZIP 保留单版本模式。

全站项目与分支选择统一到阅读上下文。代码、检索、知识、图谱及问答使用固定版本，切换时使旧请求失效，未就绪分支不借用默认分支结果。分支支持跟踪、逻辑归档和恢复，并对仍被上下文、任务或知识等引用的产物保留清理保护。

3.2 代码与知识联合检索

源码组织为携带路径、符号、行号和快照身份的片段。代码与已发布的可见知识通过统一检索服务召回、排序和展示，用户可从结果查看原文与关联范围。

关键词检索构成最小可用路径；字符相似度、外部向量模型和 CodeGraph 属于增强能力。增强能力未配置或不可用时，不应把降级结果包装为完整语义理解，也不应借用其他版本的产物。

3.3 知识管理与维护

支持知识卡创建、编辑、审核、发布、撤回、修订历史及来源关联。项目共享知识与分支专属知识独立维护，不按相同标题或正文自动合并、覆盖。

Markdown 来源按项目、分支、快照和路径组织，避免同路径跨分支覆盖。分支知识验证已迁至知识详情，保留验证结论、说明、管理权限及修订/快照绑定，知识列表支持当前分支验证状态展示与筛选。

3.4 问答、图谱与 AI 工具接入

项目问答基于检索材料提供回答与引用，并保存会话来源信息。代码图谱辅助查看结构关系和源码位置，其结论受图谱产物、语言支持和算法覆盖范围限制。

MCP 提供只读 `search_project` 工具，复用同一后端检索服务和权限边界，使网页与 AI 客户端使用一致的查询基础。平台不通过这些能力自动修改代码、执行审批或判定发布是否通过。

3.5 权限、任务与基础工程能力

具备账号、会话、项目成员权限、模型配置及系统审计等基础管理能力。仓库读取在服务端校验 `READ` 权限，维护与管理操作继续按相应权限控制。

后台任务采用数据库持久化状态，支持准备、索引及图谱等操作的状态诊断。任务中心区分普通索引任务、向量任务与分支准备任务；分支任务支持项目/分支筛选、分页历史和错误详情。

技术底座采用 Java 17 / Spring Boot、Vue 3 / TypeScript、PostgreSQL / pgvector 与 Flyway，并提供启动、诊断、部署及测试脚本。

3.6 产品收敛与界面优化

已清理不再服务主流程的遗留组件、API 包装和跨仓工程功能。跨仓退出使用带数据检查的前向迁移，存在跨仓业务数据的部署需先迁移或导出，不静默扩大知识适用范围。

分支面板保留选择、同步、一键准备、打开代码与状态摘要，高级操作折叠；知识验证与任务历史分别由知识详情和任务中心承接。主菜单统一为项目总览、联合检索、代码图谱、项目问答、知识管理、项目管理、任务中心等清楚且长度接近的名称。

04 项目优点

优点	体现	对使用与维护的意义
源码与知识协同	同一入口查找实现与相关说明，保留两类结果身份	兼顾代码定位和业务背景理解
来源可追溯	文件位置、知识修订、快照与提交信息保留在证据链中	便于核对与解释，减少只依赖生成答案的风险
多分支边界明确	固定阅读上下文，分支产物隔离，旧请求不得覆盖新选择	适配多分支维护，降低版本混淆
知识资产独立	共享和专属卡片独立维护，验证按修订与快照记录	便于维护共性知识并保留分支差异
增强能力可选	最小路径不强制依赖外部模型或 CodeGraph	可分阶段接入，控制外部依赖与使用成本
网页与 AI 接入一致	MCP 复用后端检索与权限逻辑	减少不同入口之间的规则差异
工程底座较集中	业务、任务和向量以 PostgreSQL 为主要持久化基础	当前阶段部署组件较少；规模扩大后的性能仍需验证
产品职责逐步清晰	退出非核心能力，减少重复入口并集中诊断信息	有利于后续围绕主流程维护和迭代

05 验证结果与结项依据

以下统计对应不同阶段或不同范围，不能简单相加成一次完整验收。

验证范围	已有结果	说明
跨仓清理后的前端回归	31 个测试文件、122 项通过	对应当时版本
分支界面改造后的前端回归	32 个测试文件、126 项最终通过	全量运行有一项旧权限断言失败，更新后对应路由测试单独复测通过；最终任务摘要调整的相关测试也通过
菜单命名回归	2 个测试文件、7 项通过；类型检查通过	为导航与路由的针对性复测
后端最近各测试类报告	76 个测试类，234 项中229项通过、5项跳过，无失败或错误	各类最近报告汇总，不代表最终版本重新执行了一次完整集成验收
分支任务相关后端回归	7 项通过、1 项数据库集成测试因环境条件跳过	包含访问权限与分页参数边界等测试
构建	分支改造后前端生产构建、后端运行包构建通过	菜单后续调整通过类型检查及导航测试
本地运行	更新后前端页面、后端健康检查及代理健康检查均返回 HTTP 200	已记录的本地运行结果，不等同于生产验收
数据保护	跨仓迁移前已保留数据库备份；迁移包含非空数据拒绝检查	分支面板与菜单改造未新增数据库结构变更

早期实用化验收记录还包含独立 PostgreSQL/pgvector 环境下的登录、ZIP、索引、检索、源码、本地问答和引用持久化验证。这些记录证明历史版本相应链路曾经通过，不能直接替代最新多分支版本的完整端到端复验。

当前尚未完成登录后的新增分页历史接口实际验证、最新版本完整浏览器端到端操作、真实外部模型与 CodeGraph 全链路验收及生产部署验收。当前需求中的代表性业务问题召回标准，也仍需用真实项目和人工标注样本给出量化结果。

06 待提高的部分

优先级	待提高部分	下一步建议及完成标准
P1	真实场景的检索质量缺少量化结论	建立多语言、业务术语和多分支人工标注样本，统计 Recall@10、排序质量及错误类型；形成可重复的质量报告
P1	最新版本完整业务闭环验收不足	在独立环境验证两项目各两分支的接入、准备、切换、共享/专属知识、验证保存、历史分页、问答引用和权限拒绝；保留操作证据
P1	知识长期维护机制需加强	明确知识负责人、复核周期与失效处理，跟踪待验证和过期积压；证明代码更新后的知识问题能够被发现并处理

优先级	待提高部分	下一步建议及完成标准
P1	生产运维与恢复能力未完成验收	验证数据库与受管文件一致备份、恢复演练、密钥保管、升级迁移及任务崩溃恢复；明确故障处理流程
P2	大仓库与并发场景性能未知	测量索引耗时、查询 P95、存储增长和并发任务资源；再按瓶颈优化。前端构建仍提示部分包较大，需实测首屏性能
P2	分支任务操作闭环仍有限	当前失败任务需回分支管理重新提交。若实际需要取消或专用重试，应补齐服务端状态规则和权限验证后再提供按钮
P2	模型问答与图谱能力需校准	用真实模型和 CodeGraph 验证引用、错误、超时与覆盖范围；分别展示格式合法、引用覆盖和人工事实核验，不把关系图当作运行时全链路
P2	使用文档与现行界面存在差异	更新 README、用户手册中的旧菜单及准备路径，明确历史文档与当前基线，提供一份最新首次使用指南
P3	附件内容尚未进入全文检索	若用户材料主要是 PDF/Office，可评估受控抽取与索引；补齐来源、权限和安全验证，避免上传成功被理解为内容已入库
P3	项目效益缺少用户数据	试点采集定位耗时、有效命中、引用核对成功率与知识维护成本，用反馈验证优先级，而非直接宣称效率提升比例

P1 优先补齐当前主流程的质量和可靠性；P2 根据使用规模推进；P3 以真实需求决定是否实施。已退出的跨仓、审查与门禁能力不默认恢复。

07 交付与移交

交付内容	位置或说明
后端、前端与 MCP 源码	backend、frontend、mcp-server
当前产品需求	最小产品需求
多分支与独立知识方案	项目一分支与独立共享知识
项目与分支操作说明	项目与分支代码管理
分支界面改造与验证记录	分支面板简化
启动、诊断、部署与评测材料	scripts、deploy、evaluation及相关说明
跨仓清理前备份	.runtime/backups/cross-repository-cleanup-1789465909364.dump；需纳入正式备份管理，不仅依赖工作区副本

移交时应一并确认运行环境、数据库及受管文件目录、密钥保存责任、维护负责人和后续验收安排。当前改动仍位于工作区，未因生成报告而创建正式发布标签或进行生产发布。

08 结项结论

本项目已形成以版本化源码和独立知识资产为基础的联合检索工具，具备项目接入、多分支阅读、知识维护、来源核对和辅助问答的主要能力。其突出价值是把实现与知识放到同一查找路径，并让结果的来源和适用版本更容易核对。

依据现有研发交付与验证记录，可作为阶段性研发结项成果移交。整体业务验收与生产可用性仍需在真实项目、独立验收环境及生产运行条件下确认。后续重点应放在检索质量、知识维护闭环、端到端验证和运维可靠性上。